

# Control Unit: what it is and how to explain it

## 1. The one idea you need

Every processor = **datapath** (the hardware that moves and computes data) + **control unit** (the brain that decides what the datapath does this cycle). This is the same "datapath + controller" split your textbook chapters on register-transfer design teach.

Your repo already had the whole datapath as separate parts (ALU, register file, PC, memories, immediate generator, comparator). The missing piece was the control unit, and the wiring (`simple-datapath.v`, module `riscv_core`).

The control unit reads 3 fields of the 32-bit instruction:

| Field               | Bits    | Meaning                                                   |
|---------------------|---------|-----------------------------------------------------------|
| <code>opcode</code> | [6:0]   | what <i>kind</i> of instruction (R-type, load, branch...) |
| <code>funct3</code> | [14:12] | which variant (add vs xor vs shift...)                    |
| <code>funct7</code> | [31:25] | only bit 30 matters: ADD vs SUB, SRL vs SRA               |

and outputs **control signals** (1-bit or few-bit switches):

| Signal                                                        | Steers                                                                      | Example                           |
|---------------------------------------------------------------|-----------------------------------------------------------------------------|-----------------------------------|
| <code>reg_write</code>                                        | register file write enable                                                  | 1 for ADD, 0 for STORE            |
| <code>alu_src_a</code>                                        | mux: rs1 or PC into ALU                                                     | PC only for AUIPC                 |
| <code>alu_src_b</code>                                        | mux: rs2 or immediate into ALU                                              | immediate for ADDI, loads, stores |
| <code>alu_op</code>                                           | which ALU operation                                                         | same codes as <code>alu.v</code>  |
| <code>mem_read</code> /<br><code>mem_write</code>             | data memory                                                                 | loads / stores                    |
| <code>wb_sel</code>                                           | what gets written back to <code>rd</code> : ALU, memory, PC+4, or immediate | PC+4 for JAL                      |
| <code>branch</code> , <code>jal</code> ,<br><code>jalr</code> | next-PC logic                                                               |                                   |

**It is a lookup table.** Instruction type goes in, control word comes out (the table is at the top of `control-unit.v`). Nothing more.

## 2. How one instruction flows (use this in your defence)

Take `add x3, x1, x2`:

1. PC addresses instruction memory, which returns the 32-bit word.
2. Control unit sees opcode `0110011` (R-type), so `reg_write=1`, `alu_src_b=0` (use rs2), `wb_sel=ALU`. It sees `funct3=000`, `funct7[5]=0`, so `alu_op=ADD`.
3. Register file reads x1 and x2. ALU adds them.
4. The result goes through the write-back mux to x3 at the clock edge. PC becomes PC+4.

Now `lw x5, 8(x2)`: the control unit sets `alu_src_b=1` (immediate), `mem_read=1`, `wb_sel=MEM`, `reg_write=1`. The ALU computes the address `x2 + 8`, data memory returns the word, and it is written to x5.

Branches and jumps only change the **next PC mux** (bottom of `simple-datapath.v`): `jalr` uses the ALU result, `jal` or a taken branch uses `PC + imm`, otherwise `PC + 4`.

### 3. Two subtle points examiners like

- **ADD vs SUB:** `funct7[5]` means SUB only for R-type. In `ADDI`, that bit is just the top bit of the immediate, so the decoder checks `is_rtype`. (Tested explicitly.)
- **x0 is hard-wired to zero:** that is why `jalr x0, ...` and `j label` (which is `jal x0, label`) work as "jump without saving a return address".

### 4. What was built and verified

| File                                                               | Role                                                                                                                                             |
|--------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>processor_core/datapath/control-unit.v</code>                | the control unit                                                                                                                                 |
| <code>processor_core/datapath/simple-datapath.v</code>             | <code>riscv_core</code> top level, wires all components                                                                                          |
| <code>processor_core/datapath/control-unit-tb.v</code>             | checks the control word for all 9 instruction types + every ALU variant + illegal opcode                                                         |
| <code>core-testbench.v</code>                                      | runs a 56-instruction program covering all 37 RV32I instructions implemented, compares all 32 registers to an independent Python reference model |
| <code>processor_core/tests/asm.py</code> , <code>prog_all.S</code> | mini assembler + reference model + the test program                                                                                              |
| <code>Makefile</code>                                              | <code>make test</code> runs everything; <b>fixed:</b> it was silently skipping 2 tests and still printing "ALL PASSED"                           |

Run: `make test` from the repo root (needs `iverilog`, `python3`). I also deliberately broke SUB in the control unit to confirm the core test fails, and it does.

### 5. Honest limitations (say these before they ask)

- **Single-cycle design:** one instruction per clock, the simplest correct approach. Slow

clock, but it matches your components (combinational instruction and data reads).

- **Control unit is combinational, not an FSM.** If your course or supervisor expects a *state-machine* controller (fetch, decode, execute, memory, writeback states, as in the ASM-chart chapters), that is a multi-cycle design. Ask your supervisor which they want; I can convert it.
- Not implemented: `FENCE`, `ECALL`, `EBREAK`, CSRs, the M extension (multiply/divide), misaligned accesses, pipelining, caches (those files in your repo are still empty).

## 6. A sane plan for the remaining hours

1. **(30 min)** Run `make test`, open `core.vcd` in GTKWave, and watch `pc`, `instr`, and the control signals change. Seeing it run makes everything click.
2. **(1 to 2 h)** Read `control-unit.v` top to bottom with the table in section 1 beside it. It is ~100 lines and commented.
3. **(1 h)** Hand-trace 3 instructions (an ADD, a LW, a BEQ) through the datapath as in section 2.
4. **(1 h)** Write a tiny program of your own in `prog_all.S` style, run `make core`, and explain the result.
5. Skim the textbook pages only for vocabulary (datapath, control word, RTL notation). You do not need all 74 pages for this deliverable.